

**DESIGN AND DEVELOP A WIRELESS ENVIRONMENTAL
PARAMETER MONITORING SYSTEM FOR SMART INDUSTRIES.
THE SYSTEM REMOTELY MONITORS ENVIRONMENTAL
CONDITIONS SUCH AS TEMPERATURE, HUMIDITY, AND AIR
QUALITY IN INDUSTRIAL ENVIRONMENTS.**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B – 24MER030

MOHAMMED SABEER S – 24MER031

PRAAVIN RAJAA G – 24MER039

PRASANNA T – 24MER042

DEVA PRIYAN R – 24MEL064

SATHISH KUMAR K – 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Wireless Environmental Parameter Monitoring System for Smart Industries is designed to continuously monitor important environmental conditions such as temperature, humidity, and air quality in industrial environments. The system uses sensors to collect real-time environmental data and a wireless communication module to transmit the information to a remote monitoring platform.

The collected data can be monitored remotely, allowing industries to identify abnormal environmental conditions quickly. When the measured parameters exceed predefined safety limits, the system can generate alerts to inform operators. This helps improve worker safety, equipment protection, environmental control, and industrial efficiency.

The proposed system provides a low-cost, reliable, and real-time monitoring solution suitable for smart factories, manufacturing plants, warehouses, and other industrial applications. It also supports the concept of Industry 4.0 by enabling wireless data collection and remote monitoring.

2. Problem Statement

Industrial environments require continuous monitoring of environmental conditions to maintain worker safety, equipment reliability, and production efficiency. Parameters such as temperature, humidity, and air quality may change due to machinery operation, heat generation, chemical processes, dust, gases, and poor ventilation.

Traditional monitoring methods often depend on manual inspection and periodic measurements, which may not detect sudden changes or hazardous conditions immediately. This can result in equipment overheating, poor working conditions, reduced product quality, and potential health and safety risks for workers.

Another major challenge is that conventional monitoring systems may require wired connections and local observation, making them difficult to install, maintain, and monitor from a remote location. Operators may not receive immediate information when environmental parameters exceed safe limits.

Therefore, a wireless environmental parameter monitoring system is required to continuously measure temperature, humidity, and air quality and transmit the collected data to a remote monitoring platform. The system should provide real-time information and generate alerts whenever abnormal or unsafe conditions are detected.

The proposed system aims to provide a low-cost, reliable, scalable, and real-time monitoring solution for smart industries. It can support better environmental management, early detection of hazardous conditions, preventive maintenance, improved workplace safety, and the development of Industry 4.0-based smart manufacturing systems.

3. Proposed Solution

The proposed system is a Wireless Environmental Parameter Monitoring System designed to continuously monitor environmental conditions in smart industrial environments. The system uses sensors to measure temperature, humidity, and air quality in real time.

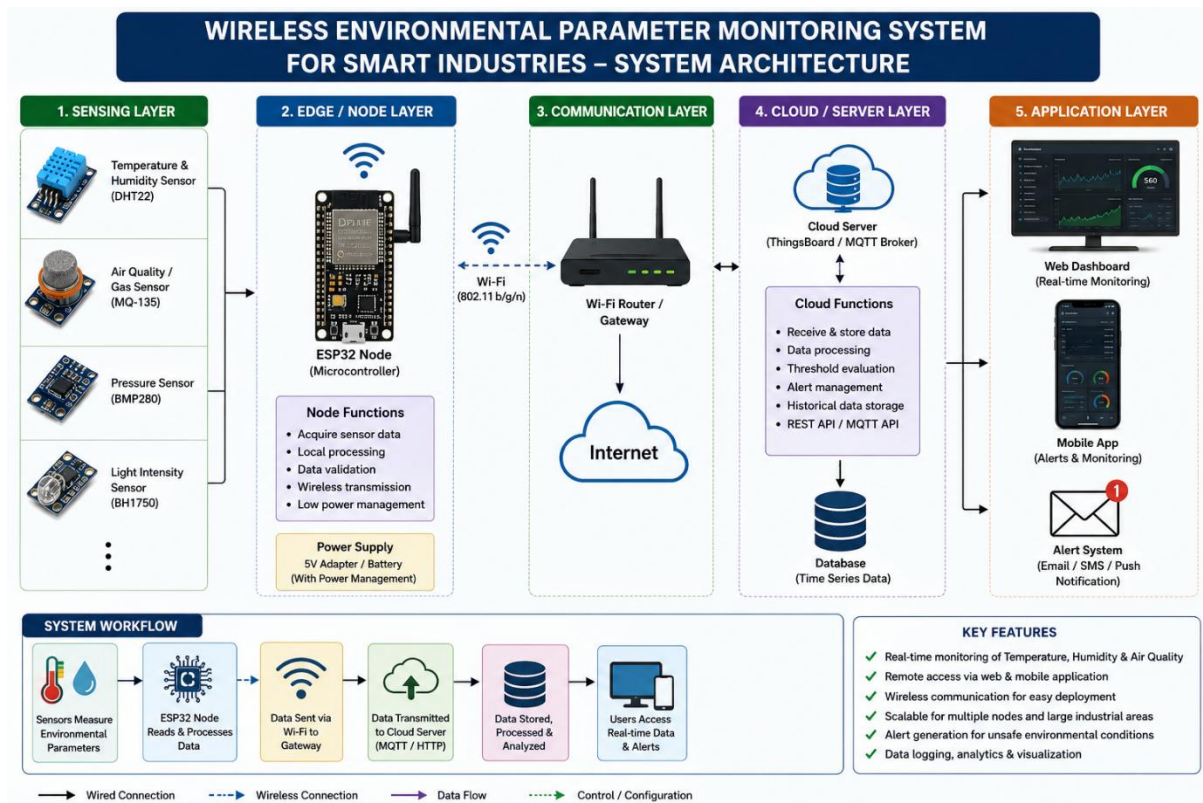
A microcontroller collects the sensor data and processes the measured values. A wireless communication module then transmits the data to a remote monitoring platform, allowing operators to observe the environmental conditions without being physically present at the monitoring location.

The system compares the measured values with predefined safety limits. If temperature, humidity, or air quality reaches an unsafe level, the system provides an alert or warning to the operator. This enables quick action before the condition causes equipment damage, production problems, or safety risks.

Main Features

- Continuous monitoring of temperature, humidity, and air quality.
- Wireless transmission of sensor data to a remote monitoring system.
- Real-time display of environmental parameters.
- Automatic detection of abnormal conditions.
- Alert generation when preset limits are exceeded.
- Reduces the need for manual environmental inspection.
- Helps improve worker safety and equipment protection.
- Supports remote monitoring and data-based decision-making.
- Suitable for smart factories, manufacturing plants, warehouses, and industrial facilities.
- Provides a foundation for Industry 4.0 and IoT-based industrial monitoring.

4. System Architecture

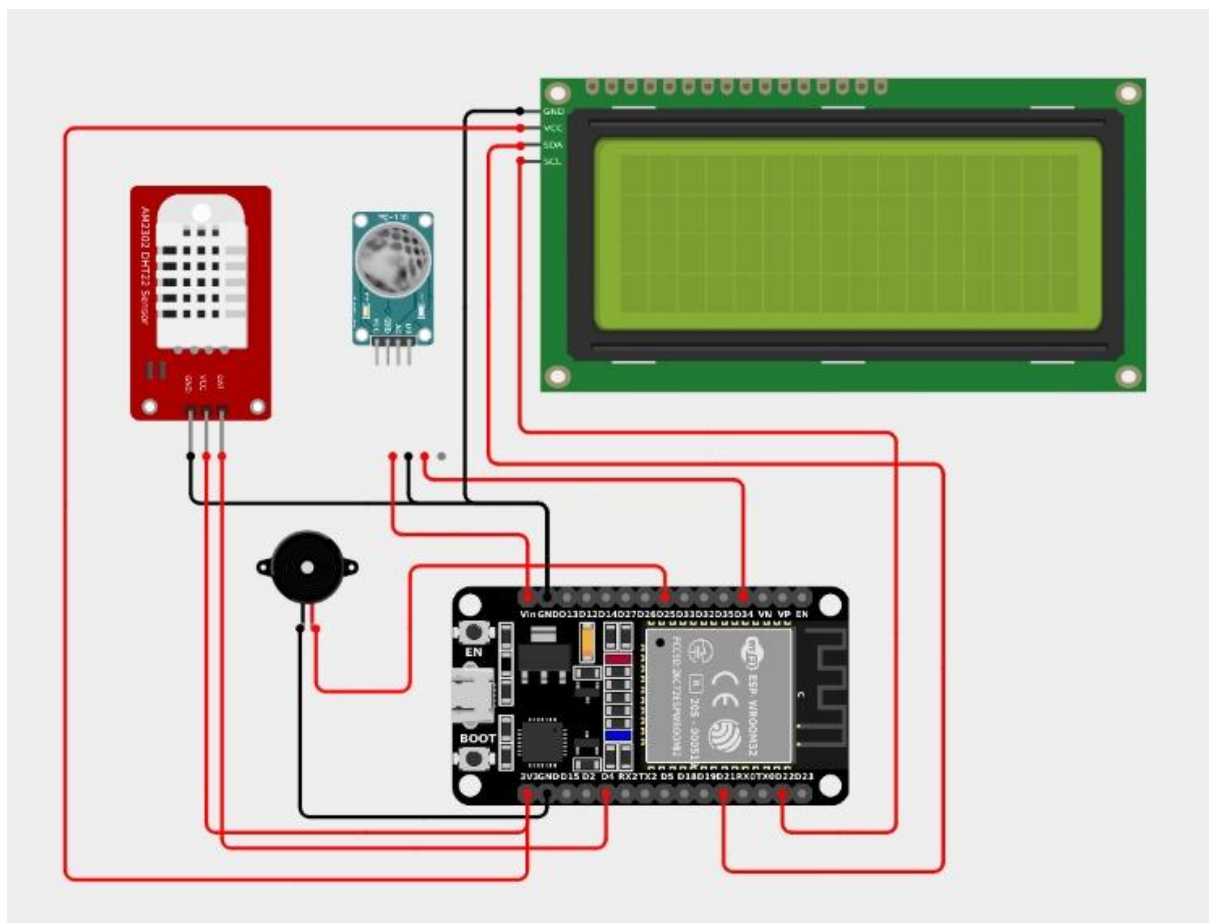


5. Circuit Table

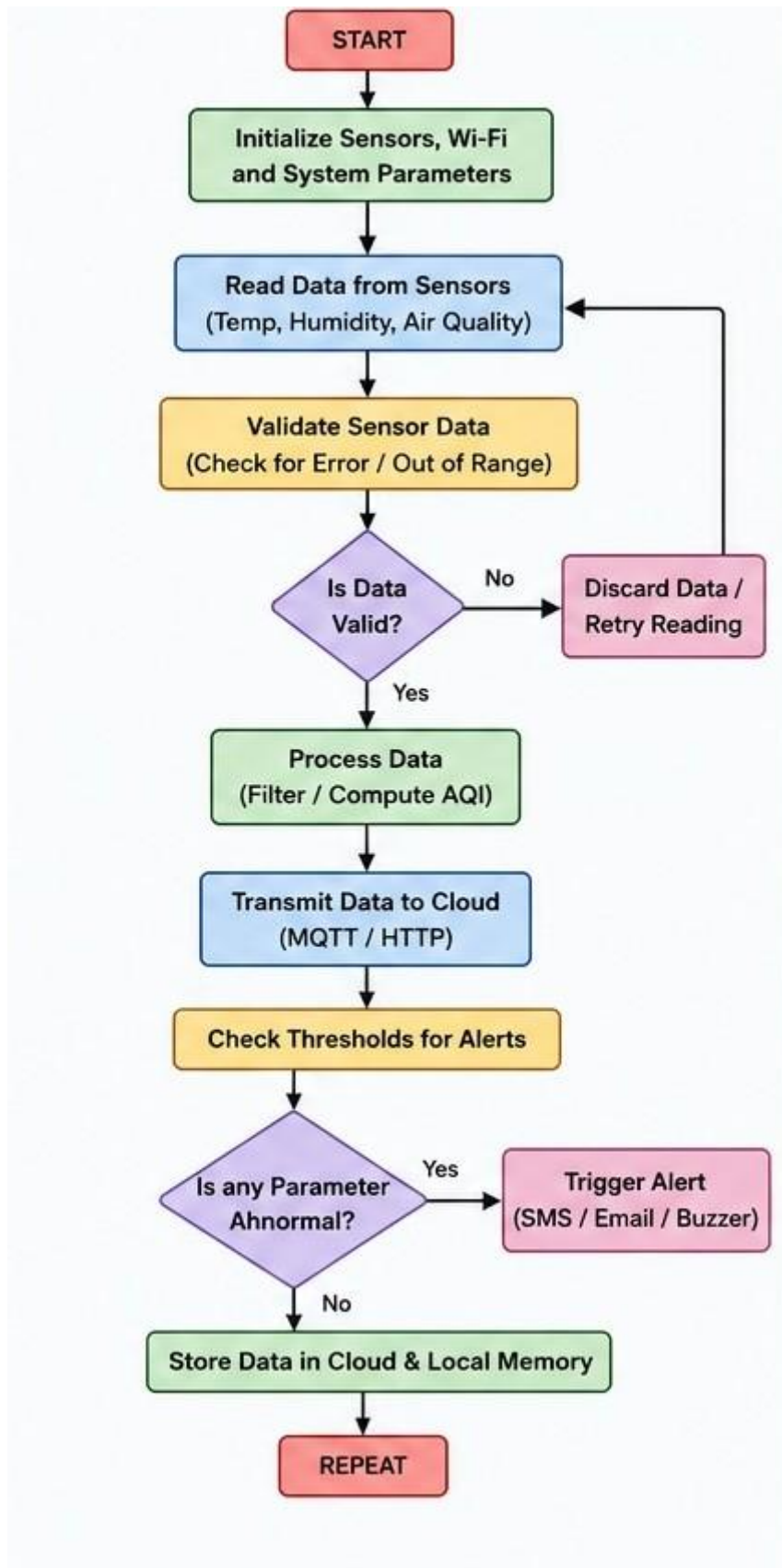
S.No.	Component	Pin/Terminal	Connection
1	Arduino/ESP32 Microcontroller	VCC	5V/3.3V Supply
2	DHT11 Sensor	VCC	5V
3	DHT11 Sensor	GND	GND
4	DHT11 Sensor	DATA	Digital Pin
5	Air Quality Sensor (MQ-135)	VCC	5V
6	MQ-135 Sensor	GND	GND
7	MQ-135 Sensor	AO	Analog Input Pin
8	ESP32/Wireless Module	VCC	3.3V/5V as required

S.No.	Component	Pin/Terminal	Connection
9	ESP32/Wireless Module	GND	GND
10	LCD/OLED Display	VCC	Supply Voltage
11	LCD/OLED Display	GND	GND
12	LCD/OLED Display	SDA/SCL	I ² C Pins
13	Buzzer	Positive (+)	Digital Output Pin
14	Buzzer	Negative (-)	GND
15	LED	Anode (+)	Digital Output through resistor
16	LED	Cathode (-)	GND

6. Circuit Design



7. Algorithm



8. Coding

```
// =====  
  
// WIRELESS ENVIRONMENTAL PARAMETER MONITORING SYSTEM  
// ESP8266 + DHT11/DHT22 + MQ135 + Buzzer + Adafruit IO  
// =====  
  
#include <ESP8266WiFi.h>  
#include "AdafruitIO_WiFi.h"  
#include <DHT.h>  
  
// =====  
  
// WIFI DETAILS  
// =====  
  
#define WIFI_SSID "HITMAN"  
#define WIFI_PASS "123456789"  
  
// =====  
  
// ADAFRUIT IO DETAILS  
// =====  
  
#define IO_USERNAME " sathishkumar_67 "  
#define IO_KEY      " aio_zGJB61OPmoc4vVW2SSfURReqoBFu "  
  
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);  
  
// =====  
  
// PIN DEFINITIONS
```

```
// =====
```

```
#define DHT_PIN 4 // D2 = GPIO4
```

```
#define MQ135_PIN A0
```

```
#define BUZZER_PIN 14 // D5 = GPIO14
```

```
// =====
```

```
// DHT SENSOR TYPE
```

```
// =====
```

```
// For DHT11
```

```
#define DHTTYPE DHT11
```

```
// For DHT22, use:
```

```
// #define DHTTYPE DHT22
```

```
DHT dht(DHT_PIN, DHTTYPE);
```

```
// =====
```

```
// ADAFRUIT IO FEEDS
```

```
// =====
```

```
AdafruitIO_Feed *temperatureFeed = io.feed("temperature");
```

```
AdafruitIO_Feed *humidityFeed = io.feed("humidity");
```

```
AdafruitIO_Feed *airQualityFeed = io.feed("air-quality");
```

```
AdafruitIO_Feed *airStatusFeed = io.feed("air-status");
```

```
// =====
```

```
// AIR QUALITY THRESHOLDS
```



```
// =====

#define GOOD_LIMIT    399
#define MODERATE_LIMIT  599

// =====
// SETUP
// =====

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("=====");
  Serial.println(" WIRELESS ENVIRONMENT MONITORING SYSTEM");
  Serial.println(" ESP8266 + DHT + MQ135 + BUZZER");
  Serial.println(" ADafruit IO");
  Serial.println("=====");

  // Start DHT
  dht.begin();

  // Buzzer
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);
```

```
// =====  
  
// CONNECT TO ADAFRUIT IO  
  
// =====  
  
Serial.println("Connecting to Adafruit IO...");  
  
io.connect();  
  
while (io.status() < AIO_CONNECTED)  
{  
  Serial.print(".");  
  delay(500);  
}  
  
Serial.println();  
Serial.println("Adafruit IO Connected!");  
Serial.println("-----");  
}  
  
// =====  
  
// LOOP  
  
// =====  
  
void loop()  
{  
  // Keep Adafruit IO connection alive  
  io.run();  
  
  // =====
```

```
// READ DHT
```

```
// =====
```

```
float temperature = dht.readTemperature();
```

```
float humidity = dht.readHumidity();
```

```
if (isnan(temperature) || isnan(humidity))
```

```
{
```

```
    Serial.println("ERROR: DHT sensor reading failed!");
```

```
    delay(2000);
```

```
    return;
```

```
}
```

```
// =====
```

```
// READ MQ135
```

```
// =====
```

```
int airQualityValue = analogRead(MQ135_PIN);
```

```
// =====
```

```
// AIR QUALITY STATUS
```

```
// =====
```

```
String airStatus;
```

```
if (airQualityValue <= GOOD_LIMIT)
```

```
{
```

```
    airStatus = "GOOD";
```

```
    digitalWrite(BUZZER_PIN, LOW);
}
else if (airQualityValue <= MODERATE_LIMIT)
{
    airStatus = "MODERATE";

    digitalWrite(BUZZER_PIN, LOW);
}
else
{
    airStatus = "POOR";

    digitalWrite(BUZZER_PIN, HIGH);
}

// =====
// SERIAL MONITOR
// =====

Serial.println();
Serial.println("----- SENSOR DATA -----");

Serial.print("Temperature    : ");
Serial.print(temperature);
Serial.println(" °C");

Serial.print("Humidity        : ");
Serial.print(humidity);
Serial.println(" %");
```

```
Serial.print("Air Quality Value : ");
```

```
Serial.println(airQualityValue);
```

```
Serial.print("Air Quality    : ");
```

```
Serial.println(airStatus);
```

```
Serial.print("Buzzer        : ");
```

```
if (airQualityValue > MODERATE_LIMIT)
```

```
{
```

```
    Serial.println("ON");
```

```
}
```

```
else
```

```
{
```

```
    Serial.println("OFF");
```

```
}
```

```
Serial.println("-----");
```

```
// =====
```

```
// SEND DATA TO ADAFRUIT IO
```

```
// =====
```

```
Serial.println("Sending data to Adafruit IO...");
```

```
temperatureFeed->save(temperature);
```

```
humidityFeed->save(humidity);
```

```
airQualityFeed->save(airQualityValue);
```

```
airStatusFeed->save(airStatus);
```

```
Serial.println("Data sent successfully!");
```

```
// =====
```

```
// WAIT
```

```
// =====
```

```
delay(5000);
```

```
}
```

9. System Working

1. **Power ON:** The system is powered using a suitable DC power supply.
2. **Sensor Detection:** The temperature and humidity sensor measures the surrounding temperature and humidity, while the air-quality sensor detects harmful gases and pollutants.
3. **Data Processing:** The microcontroller receives the sensor readings and processes them continuously.
4. **Parameter Comparison:** The measured values are compared with predefined safe limits.
5. **Wireless Transmission:** The processed environmental data is transmitted wirelessly to a remote monitoring platform.
6. **Remote Monitoring:** The operator can view the temperature, humidity, and air-quality values remotely in real time.
7. **Alert Generation:** If any parameter exceeds its safe limit, the system activates an alert through a buzzer, LED, or remote notification.
8. **Continuous Monitoring:** The system continuously repeats the measurement and transmission process, providing real-time environmental monitoring for smart industrial applications.

10. Bill of Materials

S.No.	Component	Quantity	Purpose
1	ESP32 / Arduino Board	1	Main controller and data processing
2	DHT11/DHT22 Sensor	1	Measures temperature and humidity
3	MQ-135 Air Quality Sensor	1	Detects air pollutants and harmful gases
4	Wi-Fi Module	1	Wireless data transmission (<i>if not built into controller</i>)
5	OLED/LCD Display	1	Displays sensor readings
6	Buzzer	1	Provides warning alert
7	LED	2	Indicates normal and warning conditions
8	Resistors	2-4	LED and sensor circuit protection
9	Breadboard	1	Circuit assembly
10	Jumper Wires	1 set	Electrical connections
11	DC Power Supply/USB Cable	1	Provides power to the system
12	Connecting Wires	As required	Circuit connections
13	Enclosure/Project Box	1	Protects the electronic components

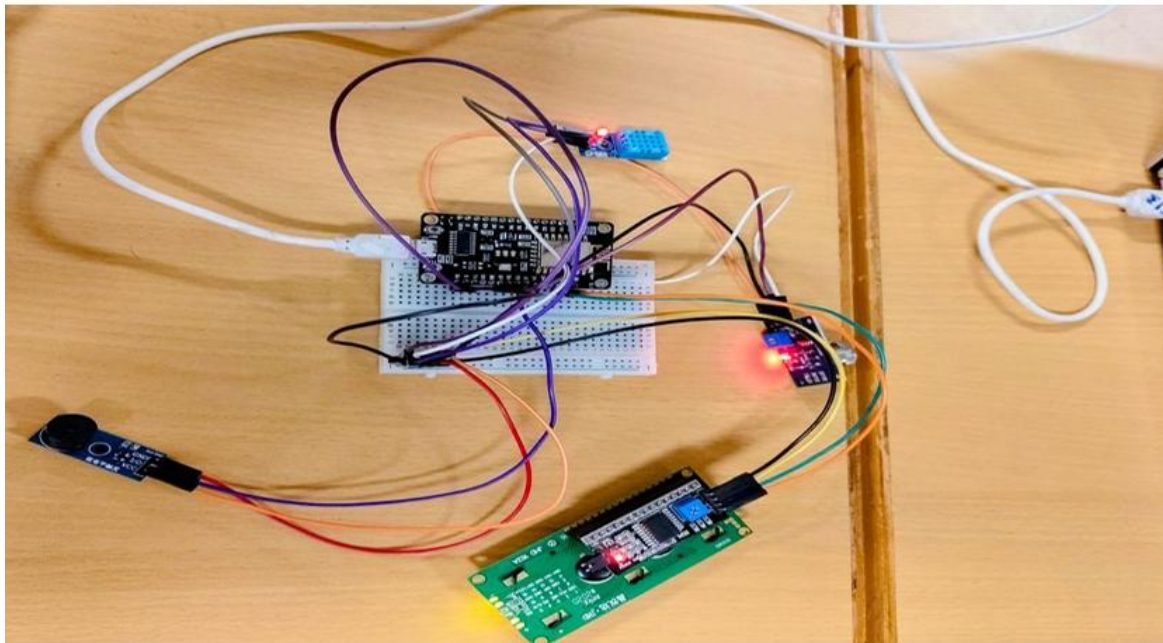
11. Prototype Implementation.

The prototype of the Wireless Environmental Parameter Monitoring System is developed using an ESP32/Arduino microcontroller, DHT11/DHT22 temperature and humidity sensor, and MQ-135 air-quality sensor. The sensors are connected to the microcontroller to collect environmental data continuously.

The microcontroller processes the sensor readings and sends the data through a wireless Wi-Fi connection to a remote monitoring platform. An LCD/OLED display can also be used to show the measured values locally.

A buzzer and LED indicators are included to provide alerts when temperature, humidity, or air-quality values exceed the predefined safe limits. The prototype is assembled on a breadboard and tested under different environmental conditions to verify sensor readings and wireless communication.

The developed prototype demonstrates the basic operation of a real-time, wireless, and remote environmental monitoring system suitable for smart industrial applications.



12. Results & Performance Analysis

Results

The developed Wireless Environmental Parameter Monitoring System successfully measured temperature, humidity, and air quality using the connected sensors. The ESP32/Arduino processed the sensor readings and transmitted the data wirelessly to the remote monitoring platform.

The system continuously displayed the environmental parameters and generated an alert through the buzzer/LED when the measured values exceeded the predefined safety limits. The prototype demonstrated reliable real-time monitoring and wireless data transmission.

Performance Analysis

- Real-time Monitoring: Environmental parameters were monitored continuously.
- Sensor Response: The sensors responded to changes in temperature, humidity, and air quality.
- Wireless Communication: Sensor data was transmitted successfully to the remote monitoring system.
- Alert System: Warning alerts were activated when unsafe conditions were detected.
- Remote Monitoring: Operators can monitor industrial environmental conditions without being physically present.
- Reliability: The prototype provided stable operation during testing.
- Overall Performance: The system is suitable for smart industries, factories, warehouses, and industrial safety applications.

Overall Result

The prototype achieved the main objective of providing a low-cost, wireless, real-time environmental monitoring and alert system for smart industrial environments.

